Tipuri de Date. Încapsulare. (specificatori de acces, constructori, static, referințe)

Alexandru Olteanu

Universitatea Politehnica Bucuresti
Facultatea de Automatică si Calculatoare, Departamentul Calculatoare
alexandru.olteanu@cs.pub.ro

OOP, 2018







Agenda

- Analiză privind tipurile de date (Java, C++)
- Încapsulare: hands-on clasa Activity din Android
- Proiectarea aplicației EduMediaPlayer pentru mobil
- Proiectarea aplicației EduRenewableEnergy pentru server

Tipuri de date

Clasificare

Tipuri de date:

- primitive
- structurate
- omogene

Tipuri de date

Tipuri de date in C/C++:

- primitive
  - int, char, float, double, void
  - signed, unsigned

Tipuri de date

Tipuri de date in C/C++:

- primitive
  - int, char, float, double, void
  - signed, unsigned
- structurate
  - struct din C
  - struct, class din C++

Tipuri de date

Tipuri de date in C/C++:

- primitive
  - int, char, float, double, void
  - signed, unsigned
- structurate
  - struct din C
  - struct, class din C++
- omogene
  - vectori (a.k.a. arrays)
  - structuri de date (implementate manual sau din STL)

Tipuri de date

Tipuri de date in Java:

- primitive:

```
float pret = 12.5;
```

- structurate - class:

```
Song song1 = new Song("Bucovina", "Mestecanis");
```

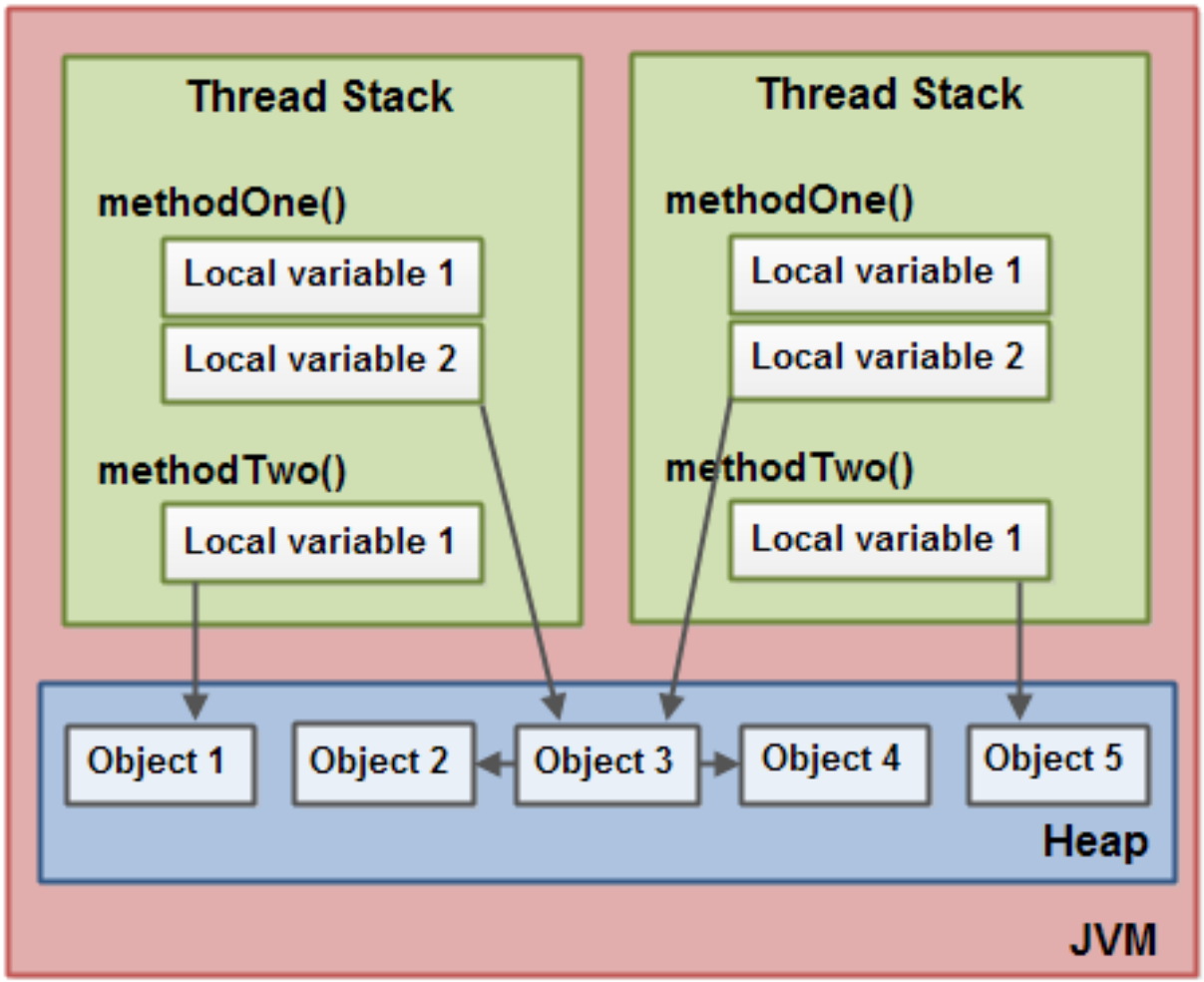
- omogene:

```
Song playlist[] = new Song[]{
  new Song("Subcarpati", "Balada Romanului"),
  new Song("Subcarpati", "Frunzulita, iarba deasa")
};
```

Tipuri de date

	definire	declarare (alocarea memoriei, constante)	inițializare (valori default)	folosire	eliberarea memoriei
primitive					
structurate					
omogene					

Alocarea memoriei



▸ Java Memory Model on Jenkov.com

Pointeri vs Referințe în C/C++

```
int x = 13, y = 14;
int *px = &x, &rx = x;
```

Pointeri vs Referințe în C/C++

```
int x = 13, y = 14;
int *px = &x, &rx = x;

int *px, &rx;                    // eroare: referintele sunt
    initializate la creare
```

Pointeri vs Referințe în C/C++

```
int x = 13, y = 14;
int *px = &x, &rx = x;

int *px, &rx;              // eroare: referintele sunt
    initializate la creare

px = &y;  rx = y;          // nu se modifica referinta, ci
    variabila referita
```

Pointeri vs Referințe în C/C++

```
int x = 13, y = 14;
int *px = &x, &rx = x;

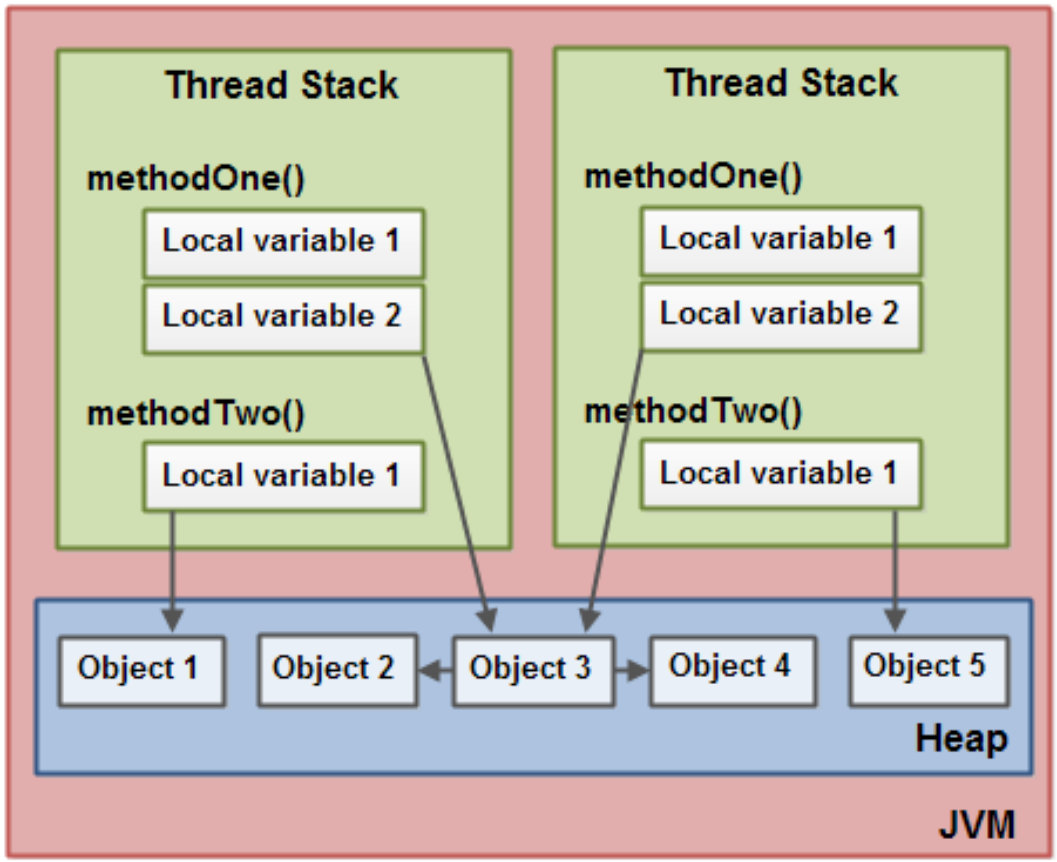
int *px, &rx;                    // eroare: referintele sunt
    initializate la creare

px = &y;   rx = y;               // nu se modifica referinta, ci
    variabila referita

px++;    rx++;                   // nu se modifica referinta, ci
    variabila referita
```

Referințe în Java

În Java avem doar referințe, nu avem pointeri



▸ Java Memory Model on Jenkov.com

Tipuri de date primitive

Tipuri de date primitive: definire

	definire	declarare	inițializare	folosire	eliberare
primitive					
structurate					
omogene					

Tipuri de date primitive: definire

Table: Tipuri primitive predefinite în Java

Tip primitiv	Dimensiune	Minim	Maxim
boolean	dimensiune variabilă	-	-
char	16	Unicode 0	Unicode 216 - 1
byte	8	-128	127
short	16	-215	215 - 1
int	32	-231	231 - 1
long	64	-263	263 - 1
float	32	IEEE754	IEEE754
double	64	IEEE754	IEEE754

fără unsigned

void nu este tip, dar este keyword

Tipuri de date primitive: declarare

	definire	declarare	inițializare	folosire	eliberare
primitive					
structurate					
omogene					

Tipuri de date primitive: declarare

În Java/C/C++: tip_de_date nume_variabilă;

Tipuri de date primitive: declarare

În Java/C/C++: tip_de_date nume_variabilă;

```
boolean favorite;
int duration;
float price;
```

Din C++11 există auto în loc de tip_de_date

Tipuri de date primitive: alocarea memoriei

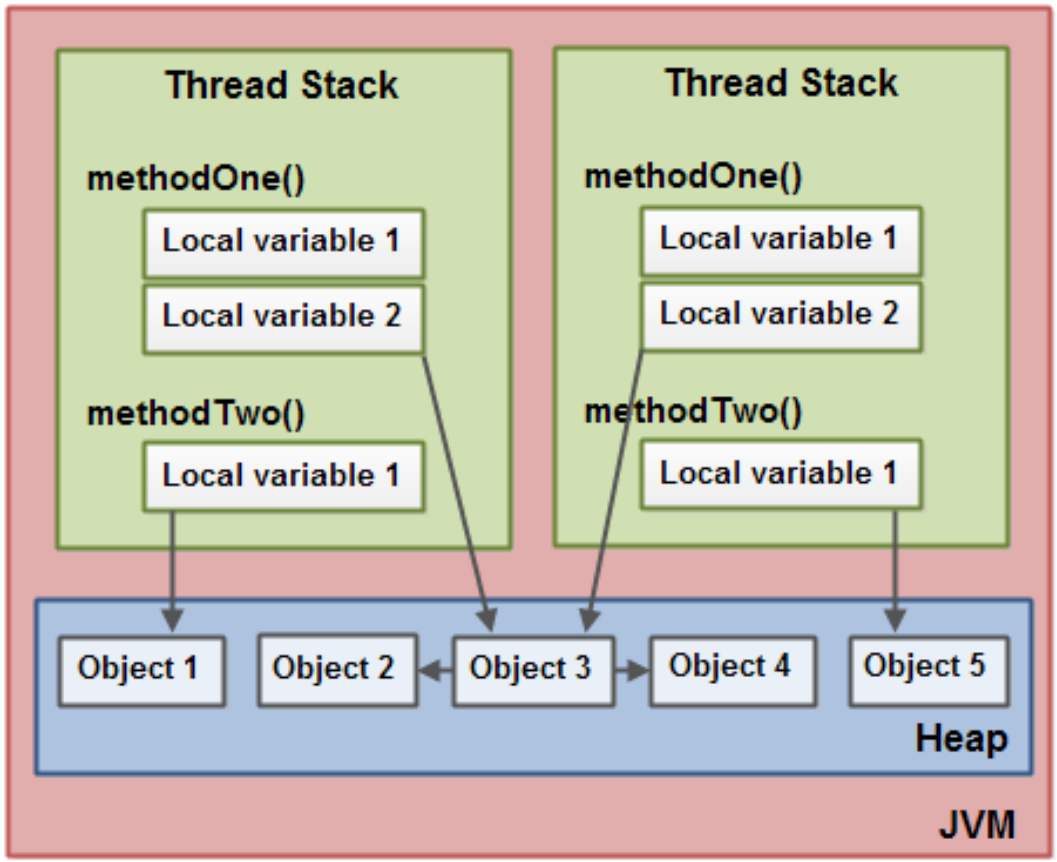
În Java, variabile primitive pot fi locale unei metode sau membrii unui obiect

```
public class Album {
  public static void main(String[] args) {
    int price;
  }
}
```

```
public class Album {
  int price;
  public static void main(String[] args) {
  }
}
```

Tipuri de date primitive: alocarea memoriei

În Java, variabile primitive pot fi locale unei metode sau membrii unui obiect



▸ Java Memory Model on Jenkov.com

Tipuri de date primitive: constante

În Java, cuvântul cheie final

```
public class Album {

    public static final int tracks = 15;
    public final int releaseYear;

    Album(int releaseYear) {
      this.releaseYear = releaseYear;
    }

}
```

Tipuri de date primitive: inițializare

	definire	declarare	inițializare	folosire	eliberare
primitive					
structurate					
omogene					

Tipuri de date primitive: inițializare

tip_de_date nume_variabilă = valoare;

Tipuri de date primitive: inițializare

tip_de_date nume_variabilă = valoare;

```
boolean favorite = false;
int duration = 480;
  // sau duration = 4*60
  // sau duration = 0x10
  // sau duration = 0b100000
double price = 1.98;
  // sau float price = 1.98f;
```

Tipuri de date primitive: valori default

variabilele locale nu sunt inițializate

```
public class Album {
  public static void main(String[] args) {
    int price;
    price++; // eroare de compilare
  }
}
```

Tipuri de date primitive: valori default

variabilele locale nu sunt inițializate

```
public class Album {
  public static void main(String[] args) {
    int price;
    price++; // eroare de compilare
  }
}
```

membrii claselor sunt inițializați cu 0 sau null, în funcție de tip

```
public class Album {
  int price;
  public static void main(String[] args) {
    price++; // bad programming style
  }
}
```

▸ Primitive Data Types doc

Tipuri de date primitive: folosire

	definire	declarare	inițializare	folosire	eliberare
primitive					
structurate					
omogene					

Tipuri de date primitive: folosire

```
teamSize=5;
teamSize++;
Player[] team = new Player[teamSize];
```

Tipuri de date primitive: eliberarea memoriei

	definire	declarare	inițializare	folosire	eliberare
primitive					
structurate					
omogene					

Tipuri de date primitive: eliberarea memoriei

în Java Garbage Collector:

- tipurile de date primitive vor fi eliberate odată cu obiectul sau vectorul care le conține

Tipuri de date structurate

struct și class în C/C++

struct în C grupează mai multe variabile denumite membrii

```
struct Produs {
  int pret;
  string descriere;
}
```

struct și class în C/C++

struct în C grupează mai multe variabile denumite membrii

```
struct Produs {
  int pret;
  string descriere;
}
```

struct și class în C++ grupează membrii și metode

```
struct Cumparator {
  string nume;
  int bani;
  void cumpara(Produs camasa) {
    bani -= camasa.pret;
  }
}
```

Class vs Object

În C++/Java avem:

- clase: tipuri de date structurate, tipare pe baza cărora se pot crea obiecte.
- obiecte: instanțe ale claselor.

În Java: everything is an object

"everything is an object":

- cu excepția primitivelor și referințelor, care există în relație cu un obiect
- toate clasele (predefinite și definite de programator) derivă din java.lang.Object

Tipuri de date structurate în Java: definire

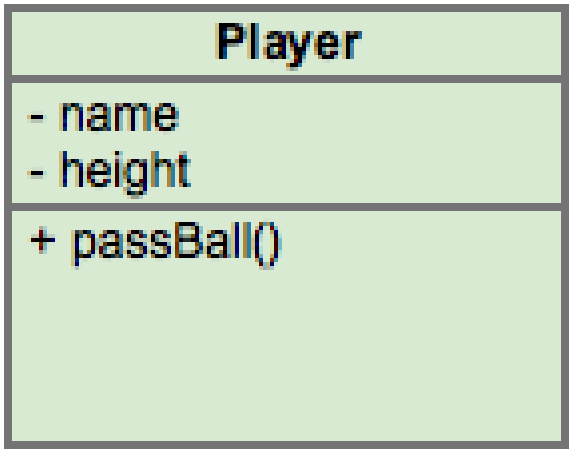
	definire	declarare	inițializare	folosire	eliberare
primitive					
structurate					
omogene					

Tipuri de date structurate: definire

Clasele conțin:

- membrii: variabile membru (câmpuri) și proprietăți, primitive sau structurate (vezi agregare)
- metode: funcții membru

Un exemplu



Specificatori de acces

- public - permite acces complet din exteriorul clasei curente
- private - limitează accesul doar în cadrul clasei curente
- protected - limitează accesul doar în cadrul clasei curente și al tuturor descendenților ei (conceptul de descendență sau de moștenire va fi explicat cursul următor)
- (default) - în cazul în care nu este utilizat explicit nici unul din specificatorii de acces de mai sus, accesul este permis doar în cadrul pachetului (package private). Atenție, nu confundați specificatorul default (lipsa unui specificator explicit) cu vreunul din ceilalți specificatori!

Specificatori de acces: exemplu

```
class GeneratorNume {
  private String[] vocabular = {new String("Ana"), new
      String("Mihai"), new String("Robert")};
  private Random randomGenerator = new Random();

  public String genNume() {
    return vocabular[randomGenerator.nextInt(3)];
  }
}
...
GeneratorNume gn = new GeneratorNume();
System.out.println(gn.genNume());
```

Constructori: Default Constructor

Default constructor (no-arg constructor): provides the default values to the object like 0, null etc. depending on the type

```
public class Magazin {
    private String brand;

    public Magazin() {
      brand = "de inchiriat";
    }
}
```

Constructori: Parameterized Constructor

Parameterized constructor: provide different values to the distinct objects

```
public class Magazin {
    private String brand;

    public Magazin(String brand) {
      this.brand = brand;
    }
}
```

Constructori: Constructor Overloading

A class can have any number of constructors that differ in parameter lists.The compiler differentiates these constructors by taking into account the number of parameters in the list and their type

```
public class Magazin {
    private String brand;
    private Integer suprafata;

    public Magazin(Integer suprafata) {
      this.suprafata = suprafata;
      this.brand = "de inchiriat";
    }

    public Magazin(Integer suprafata, String brand) {
      this.suprafata = suprafata;
      this.brand = brand;
    }
}
```

Constructori: Copy Constructor

In Java, there is no Copy Constructor per-se, but there are many ways to copy the values of one object into another:

- By constructor
- By assigning the values of one object into another
- By clone() method of Object class

```
public class Training {
    private int pushups;
    private int crunches;

    public Training(Training otherTraining) {
      this.pushups = otherTraining.pushups;
      this.crunches = otherTraining.crunches;
    }
}
```

static

static desemnează membrii și metode independente de instanțele clasei

```
public class BasketBall {
  public static int size = 12;
  private String color;

  Ball(String color) {
    this.color = color;
  }
}
```

static vs non-static

Primim eroare la compilare:
"Cannot make a static reference to the non-static field count"

```
public class TestApp {
        int count;
        public static void main(String[] args){
                count++;
        }
}
```

pentru că ceea ce se petrece într-o funcție statică ar trebui să poată funcționa chiar dacă obiectul a fost instanțiat sau nu; în acest caz, count nu există fără o instanță

static vs non-static

Dacă count chiar nu este static, o posibilă rezolvare ar fi:

```
public class TestApp {
        int count;
        public static void main(String[] args){
                TestApp app = new TestApp();
                app.incrementCount();
        }
        public void incrementCount() {
                count++;
        }
}
```

Tipuri structurate predefinite

Toate tipurile primitive au clase wrapper predefinite (e.g. Integer, Long, Float, Boolean etc.)

- exemplu: Integer height = new Integer(180);
- datele primitive pot să acționeze ca obiecte (e.g. să facă parte dintr-un ArrayList)
- oferă metode utilitare: Integer height = Integer.valueOf("180");

Tipuri de date structurate: declarare

	definire	declarare	inițializare	folosire	eliberare
primitive					
structurate					
omogene					

Tipuri de date structurate: declarare

La declararea fără inițializare se creează o referință nulă:

```
Magazin spatiuDeInchiriat;
```

Se alocă spațiu pe heap și se apelează un constructor la inițializare:

```
Magazin magazinSport = new Magazin("Articole Sportive");
```

Tipuri de date structurate: constante

```
public class Boxer {
    public final int birthYear;
    public String name;

    Boxer(String name, int birthYear) {
      this.name = name;
      this.birthYear = birthYear;
    }
}
...
final Boxer champion = new Boxer("Cassius Clay", 1942);
```

Tipuri de date structurate: constante

Variabila referință final permite modificarea internă a obiectului către care indică:

```
public class Boxer {
    public final int birthYear;
    public String name;

    Boxer(String name, int birthYear) {
      this.name = name;
      this.birthYear = birthYear;
    }
}
...
final Boxer champion = new Boxer("Cassius Clay", 1942);
champion.name = "Muhammad Ali";    // ok
```

Tipuri de date structurate: constante

O încercare nouă de a asigna o variabilă final rezultă în eroare de compilare

```
public class Boxer {
    public final int birthYear;
    public String name;

    Player(String name, int birthYear) {
      this.name = name;
      this.birthYear = birthYear;
    }
}
...
final Boxer champion = new Boxer("Cassius Clay", 1942);
champion = new Boxer("Joe Frazier", 1952);   // eroare
champion.birthYear = 2016;                   // eroare
```

Immutable objects

Immutable objects: toate atributele unui obiect admit o unică inițializare:

```
public class Ball {
  public final String color;
  public Ball(String color) {
    this.color = color;
  }
}
Ball basketball = new Ball("brown");
```

Immutable objects

Immutable objects: toate atributele unui obiect admit o unică inițializare:

```
public class Ball {
  public final String color;
  public Ball(String color) {
    this.color = color;
  }
}
Ball basketball = new Ball("brown");
```

Așa sunt, spre exemplu, obiectele de tip String sau Integer

Tipuri de date structurate în Java: inițializare

	definire	declarare	inițializare	folosire	eliberare
primitive					
structurate					
omogene					

Tipuri de date structurate: inițializare

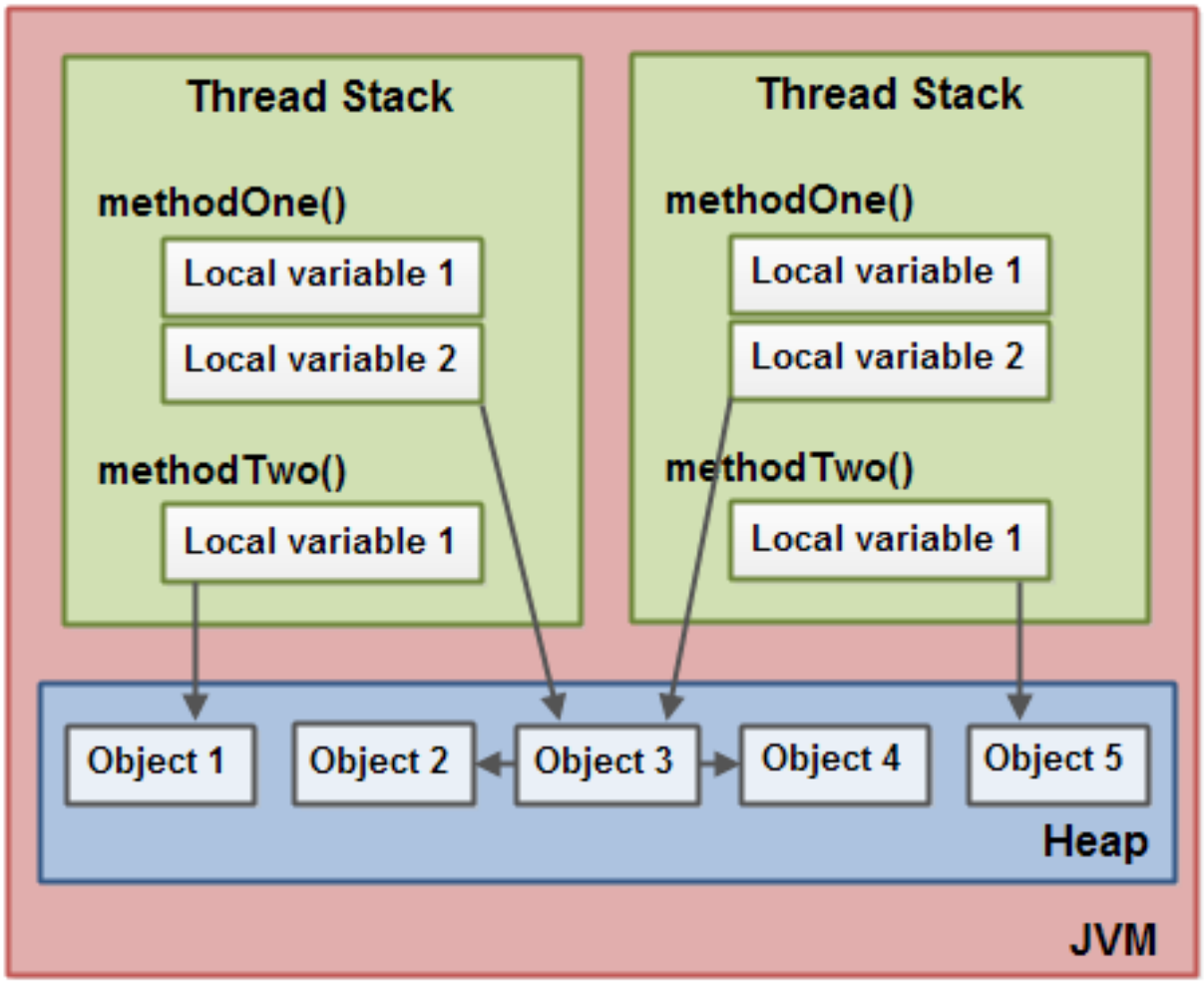
La declararea fără inițializare se creează o referință nulă:

```
Magazin spatiuDeInchiriat;
```

Se alocă spațiu pe heap și se apelează un constructor la inițializare:

```
Magazin magazinGol = new Magazin();
Magazin magazinSport = new Magazin("Articole Sportive");
```

Tipuri de date structurate: alocarea memoriei



▸ Java Memory Model on Jenkov.com

Tipuri de date structurate în Java: alocarea memoriei

Două referințe la același obiect: se modifică obiectul

```
public Arena {
  public int sits;
  Arena(int sits) {
    this.sits = sits;
  }
}

Arena arenaNationala = new Arena(55000);
stadionulNational = arenaNationala;
stadionulNational.sits += 600;
System.out.println(arenaNationala.sits); // 55000 ?
    55600
```

Tipuri de date structurate: folosire

	definire	declarare	inițializare	folosire	eliberare
primitive					
structurate					
omogene					

Tipuri de date structurate: comparații

== tests for reference equality (whether they are the same object)
.equals() tests for value equality (whether they are logically "equal")

```
String name1 = new String("Michael"), name2 = name1,
    name3 = new String("Michael");
System.out.println(name1==name2);
System.out.println(name1==name3);
System.out.println(name1.equals(name3));
```

Tipuri de date structurate: transferul parametrilor [1]

Transferul parametrilor la apelul funcțiilor este crucial pentru o funcționare corectă:

- pentru tipurile primitive se transferă prin COPIERE pe stivă: orice modificare în funcția apelată NU VA FI VIZIBILĂ în urma apelului.

[1] Adriana Drăghici, "Curs POO - Seria CA"

Tipuri de date structurate: transferul parametrilor [1]

Transferul parametrilor la apelul funcțiilor este crucial pentru o funcționare corectă:

- pentru tipurile primitive se transferă prin COPIERE pe stivă: orice modificare în funcția apelată NU VA FI VIZIBILĂ în urma apelului.
- pentru tipurile definite de utilizator și instanțe de clase în general, se COPIAZĂ REFERINȚA pe stivă: referința indică spre zona de memorie a obiectului, astfel că schimbările asupra câmpurilor vor fi vizibile după apel, dar reinstanțieri (expresii de tipul: p = new Player()) în apelul funcției și modificările făcute după ele, NU VOR FI VIZIBILE după apel, deoarece ele modifică o copie a referinței originale.

[1] Adriana Drăghici, "Curs POO - Seria CA"

Tipuri de date structurate: eliberarea memoriei

	definire	declarare	inițializare	folosire	eliberare
primitive					
structurate					
omogene					

Tipuri de date structurate: eliberarea memoriei

Garbage Collector:

- Când un obiect nu mai este folosit, Garbage Collector revendică spațiul său de memorie de pe Heap pentru a fi refolosit

▸ Java Garbage Collection Basics - Oracle

Tipuri de date omogene

Tipuri de date omogene: definire

	definire	declarare	inițializare	folosire	eliberare
primitive					
structurate					
omogene					

Tipuri de date omogene în Java

- Arrays: obiecte care stochează mai multe variabile:
  - de același tip
  - subclase ale aceluiași tip
- Pot conține primitive sau referințe

```
Player[] team = new Player[maxTeamSize];
int[] points = {15, 12, ...};
```

Tipuri de date omogene [2]

Collections (Containers): obiecte care grupează mai multe elemente oferind posibilitatea de stocare, interogare, manipulare (sortare etc.) și agregare a datelor.
Față de Arrays, oferă o funcționalitate mai complexă (pot fi mutable sau immutable, type safe etc.). Pentru mai multe detalii: Effective Java, Second Edition, pag. 119.

```
List<Player> team = new ArrayList<Player>();
```

▸ Oracle tutorial on Collections

[2] Adriana Drăghici, "Curs POO - Seria CA"

Tipuri de date omogene: definire

Puteți implementa propriile colecții pentru:

- a extinde și particulariza o anumită funcționalitate
- a incerca obținerea de performanțe mai bune

Tipuri de date omogene: declarare

	definire	declarare	inițializare	folosire	eliberare
primitive					
structurate					
omogene					

Tipuri de date omogene: declarare [3]

```
int[] x;
int y[]; // corect, dar nu se recomanda d.p.d.v coding
    style
int z = new int[9];
int[] points = new int[]; // eroare la compilare

String[] names = new String[10];

int [][] matrix = new int[10][]; // corect, de ce?
```

[3] Adriana Drăghici, "Curs POO - Seria CA"

Tipuri de date omogene: alocarea memoriei [4]

- În `Player[] team = new Player[5]` nu se creează obiecte Player!
- `new Type[capacity]` creează în **Heap** un obiect array.
- Elementele unui array nu sunt create automat, doar primitivele primesc valori default.

[4] Adriana Drăghici, "Curs POO - Seria CA"

Tipuri de date omogene: alocarea memoriei [5]

Câte obiecte se creează în heap prin codul următor:

```
int [][] a = {{1,2,3}, {4,5}};
```

[5] Adriana Drăghici, "Curs POO - Seria CA"

Tipuri de date omogene: alocarea memoriei [5]

Câte obiecte se creează în heap prin codul următor:

```
int [][] a = {{1,2,3}, {4,5}};
```

3 obiecte (în ordinea creării): un array de int[], un array {1,2,3}, un array {4,5}

[5] Adriana Drăghici, "Curs POO - Seria CA"

Tipuri de date omogene: constante [7]

- Nu folosiți `public static final` pentru array-uri
  - array-ul e final, însă obiectele din el pot fi modificate → security hole!
  - folosiți colecții nemodificabile[6]
- Nu puteți întoarce `{1,2,3}`, ci `new int[]{1,2,3}`

[6] exemplu Chapter 4, pg 70 in Joshua Bloch. 2008. Effective Java (2nd Edition) (The Java Series) (2 ed.). Prentice Hall

[7] Adriana Drăghici, "Curs POO - Seria CA"

Tipuri de date omogene: inițializare

	definire	declarare	inițializare	folosire	eliberare
primitive					
structurate					
omogene					

Tipuri de date omogene: inițializare

```
int[] y = {1, 2};
int z[] = new int[]{1,2,4};

Player[] team = {new Player(), new Player()};
```

Tipuri de date omogene: folosire

	definire	declarare	inițializare	folosire	eliberare
primitive					
structurate					
omogene					

Tipuri de date omogene: folosire [8]

```
int[] myNums = new int[10];
myNums[0] = 2;
System.out.println(myNums[0]); // 2
System.out.println(myNums[1]); // 0
System.out.println(myNums[10]); // java.lang.
    ArrayIndexOutOfBoundsException: 10

String[] words = new String[10];
System.out.println(words[9]) // null

for (int i = 0; i < words.length; i++)
    words[i] = "" + i;
```

[8] Adriana Drăghici, "Curs POO - Seria CA"

Arrays: utilitare [9]

java.util.Arrays oferă metode pentru:

- transformare în String
- copiere, umplere
- sortare, căutare binară
- equals, hashcode

```
System.out.println(myNums);
// [I@161e14f0]
System.out.println(Arrays.toString(myNums));
// [2, 0, 0, 0, 0, 0, 0, 0, 0, 0]
int[] myNums2 = Arrays.copyOf(myNums, 5);
myNums[2] = 7;
System.out.println(Arrays.toString(myNums2));
// [2, 0, 0, 0, 0]
```

[9] Adriana Drăghici, "Curs POO - Seria CA"

Tipuri de date omogene: eliberarea memoriei

	definire	declarare	inițializare	folosire	eliberare
primitive					
structurate					
omogene					

Tipuri de date omogene: eliberarea memoriei

Garbage Collector:

- Când un obiect nu mai este folosit, Garbage Collector revendică spațiul său de memorie de pe Heap pentru a fi refolosit

▸ Java Garbage Collection Basics - Oracle

Încapsulare

Încapsulare

Definition

Încapsularea este proprietatea claselor de obiecte de:

- a grupa datele și metodele aplicabile asupra datelor
- a proteja accesul la acestea față de utilizarea eronată

Încapsulare

Hands-on Activity din Android

Getter și Setter

O proprietate este un membru căruia i se atașează două metode ce îi pot expune sau modifica starea:

```
public class Employee {
    private String name;

    public String getName() {
      return points;
    }
    public void setName(String name) {
      this.name = name;
    }
}
```

Getter și Setter: motive

Pot exista nivele de acces diferite pentru setter și getter (sau chiar unul să nu existe)

```
public class Employee {
    private int salary;

    public int getSalary() {
      return salary;
    }
    protected void setSalary(int salary) {
      this.salary = salary;
    }
}
```

Getter și Setter: motive

Se poate ascunde reprezentarea internă:

```
public class Employee {
    private String street;
    private int number;
    private String city;

    public int getAddress() {
      return street+", "+number+", "+city;
    }
}
```

Getter și Setter: motive

Se pot face validări:

```
public class Employee {
    private String email;

    public void setEmail(String email) {
      if (!EmailChecker.isValid(email)) {
        System.out.println(email);
      } else {
        this.email = email;
      }
    }
}
```

Getter și Setter: motive

Se pot face conversii:

```
public class Employee {
    private float height;

    public void setHeight(float height, String measure)
        {
      if (!measure.equals("m")) {
        this.height = height / 0.3048;
      } else {
        this.height = height;
      }
    }
}
```

Protejarea datelor

Quiz: ganditi-va la un exemplu de clasa in care o parte a implementarii sa fie protejata de access din afara clasei

Aplicatii

QuickNote

Daca dati raspuns la minim 80% din GoogleForms pe care vi le propun primiti bonus 0.5p
Raspunsurile care nu respecta un limbaj decent sunt descalificate
Raspunsurile care nu au completat un user moodle valid sunt descalificate

Proiectarea aplicatiei EduMediaPlayer

- Requirements
- Design

Proiectarea aplicatiei EduRenewableEnergy

- Requirements
- Design

De citit

Este util să citiți textul laboratorului înainte de laborator:

- Organizarea surselor și controlul accesului
- Lab 2: Constructori și referințe